

A Randomized Algorithm for Single-Source Shortest Path on Undirected Real-Weighted Graphs

Ran Duan

*Institute for Interdisciplinary Information Sciences
Tsinghua University
Beijing, China
duanran@mail.tsinghua.edu.cn*

Xinkai Shu

*Department of Computer Science
The University of Hong Kong
Hong Kong, China
xkshu@cs.hku.hk*

Jiayi Mao

*Institute for Interdisciplinary Information Sciences
Tsinghua University
Beijing, China
mjoy22@mails.tsinghua.edu.cn*

Longhui Yin

*Institute for Interdisciplinary Information Sciences
Tsinghua University
Beijing, China
ylh21@mails.tsinghua.edu.cn*

Abstract—In undirected graphs with real non-negative weights, we give a new randomized algorithm for the single-source shortest path (SSSP) problem with running time $O(m\sqrt{\log n} \cdot \log \log n)$ in the *comparison-addition* model. This is the first algorithm to break the $O(m + n \log n)$ time bound for real-weighted sparse graphs by Dijkstra's algorithm with Fibonacci heaps. Previous undirected non-negative SSSP algorithms give time bound of $O(m\alpha(m, n) + \min\{n \log n, n \log \log r\})$ in comparison-addition model, where α is the inverse-Ackermann function and r is the ratio of the maximum-to-minimum edge weight [Pettie & Ramachandran 2005], and linear time for integer edge weights in RAM model [Thorup 1999]. Note that there is a proposed complexity lower bound of $\Omega(m + \min\{n \log n, n \log \log r\})$ for hierarchy-based algorithms for undirected real-weighted SSSP [Pettie & Ramachandran 2005], but our algorithm does not obey the properties required for that lower bound. As a non-hierarchy-based approach, our algorithm shows great advantage with much simpler structure, and is much easier to implement.

Index Terms—shortest path, randomized algorithm

I. INTRODUCTION

Shortest path is one of the most fundamental problems in graph theory, and its algorithms lie at the core of graph algorithm research. In a graph $G = (V, E, w)$ with $m = |E|$, $n = |V|$ and non-negative edge weight $w : E \rightarrow \mathbb{R}_{\geq 0}$, single-source shortest path (SSSP) problem asks for the distances from a given source $s \in V$ to all other vertices. Dijkstra's algorithm [1] computes the distances $\text{dist}(s, u)$ by dynamic programming. For each vertex u , it maintains a temporal distance $d(u)$, which represents the shortest path from s to u only passing through the vertices in current S , where S is the set of *visited* vertices during algorithm process. In each round of iteration it selects vertex u with the smallest $d(u)$ from the *unvisited* nodes. Finally when $S = V$, $d(u) = \text{dist}(s, u)$ for all vertex u . Advanced data structures with amortized $O(1)$ time for insertion and decrease-key, and $O(\log n)$ for extract-min, called Fibonacci heap [2] and relaxed heap [3], make the time bound for Dijkstra's algorithm to $O(m + n \log n)$. This time

bound is in the *comparison-addition* model where only comparison and addition operations on edge weights are allowed and considered as unit-time operations, which is the most common model for real number inputs. For undirected graphs, Pettie and Ramachandran [4] proposed an SSSP algorithm with running time $O(m\alpha(m, n) + \min\{n \log n, n \log \log r\})$ in the comparison-addition model, where α is the inverse-Ackermann function and r bounds the ratio of any two edge weights. However, no SSSP algorithm faster than $O(m + n \log n)$ has been found for real-weighted graphs without ratio constraints, both for undirected and directed graphs.

A byproduct of Dijkstra's algorithm is the sorting of all vertices by their distances from s , but the lower bound of $\Omega(n \log n)$ lies for comparison-based sorting algorithms. Researchers used to believe that this sorting bottleneck existed for many graph problems, and breaking this bottleneck is an important and interesting direction. Yao [5] gave a minimum spanning tree (MST) algorithm with running time $O(m \log \log n)$, citing an unpublished result of $O(m\sqrt{\log n})$ by Tarjan. The current best results for MST are the randomized linear time algorithm [6], the deterministic $O(m\alpha(m, n))$ -time algorithm [7], and a deterministic algorithm with proven optimal (but unknown) complexity [8]. In the bottleneck path problem, we want to find the path maximizing the minimum edge weight on it between two vertices. Gabow and Tarjan [9] gave an $O(m \log^* n)$ -time algorithm for s-t bottleneck path problem in directed graphs, which was later improved to randomized $O(m\beta(m, n))$ time [10]. For single-source all-destination bottleneck path problem in directed graphs, there is a recent result of $O(m\sqrt{\log n})$ -time randomized algorithm by Duan et al. [11]. For single-source nondecreasing path problem, Virginia Williams [12] proposed an algorithm with time bound $O(m \log \log n)$. All the results above are comparison-based though, the techniques in these works, such as local construction or divide-and-conquer approach, hardly works for the shortest path problem. Therefore it remains how

to break the sorting bottleneck for SSSP.

A. Our Results

In this paper we propose the first SSSP algorithm for undirected real-weighted graphs that breaks the sorting bottleneck.

Theorem 1. *In an undirected graph $G = (V, E, w)$ with nonnegative edge weights $w : E \rightarrow \mathbb{R}_{\geq 0}$, there is a comparison-addition based Las-Vegas randomized algorithm that solves the single-source shortest path problem in $O(m\sqrt{\log n} \cdot \log \log n)$ time, in which the results are always correct and it can achieve this time bound with high probability. The time complexity can be improved to $O(\sqrt{mn \log n} + n\sqrt{\log n \log \log n})$ when $m = \omega(n)$ and $m = o(n \log n)$.*

Note that there is a (worst-case) lower bound of $\Omega(m + \min\{n \log n, n \log \log n\})$ in [4] for “hierarchy-based” algorithms for undirected real-weighted SSSP, but our algorithm is randomized and not hierarchy-based. See Remark 2 for discussions.

a) Technical Overview: The bottleneck of Dijkstra-based algorithm is the priority queue. For this reason, we only add a fraction of vertices into the priority queue. As in many works on distance oracles or spanners, we sample a subset of vertices R , and the heap is only for vertices in R , then we “bundle” every other vertex v to its nearest vertex in R , which is called $b(v)$. Then define $\text{Ball}(v)$ to be the set of vertices closer than $b(v)$ to v . Since the algorithm doesn’t know the correct order of most vertices on a shortest path, relaxing only neighbours as in Dijkstra’s algorithm doesn’t work. So when popping a vertex $u \in R$ from the heap, we also deal with vertices v which are bundled to u . In an undirected graph, this also implies that $|\text{dist}(s, u) - \text{dist}(s, v)|$ is not large. Here we relax v from vertices in $\text{Ball}(v)$ and their neighbors, then from v we relax neighbors of v and vertices in their balls. (To make it easier to describe, we first change the graph to a constant-degree graph with $O(m)$ vertices.) Details of the algorithm will be discussed in Section III, as well as the analysis of correctness and running time. The detailed construction of bundles so that the algorithm can achieve the time bound w.h.p. will be introduced in Section IV. The improvement of time complexity by relaxing the constant-degree constraint will be discussed in Section V.

B. Other Related Works

Existence of algorithms better than $O(m + n \log n)$ for real-weighted SSSP has been open for long. Pettie and Ramachandran’s algorithm [4] works better than $O(n \log n)$ if the ratio between maximum and minimum edge weights is not very large. For the integer-weighted case, random access machine (RAM) model is usually adopted, where multiplications, shifts and Boolean operations on edge weights are allowed. There are many works on improving heaps and SSSP algorithms on RAM model with integer weights [13], [14], [15], [16], [17], [18], [19]. Finally Thorup gave a linear-time algorithm for undirected graphs [20] and $O(m + n \log \log \min\{n, C\})$ for

directed graphs [21] where C is the maximum edge weight. Recently, almost linear time $O(m^{1+o(1)} \log C)$ algorithms for SSSP with negative weights are also discovered [22], [23].

All-pair shortest path (APSP) problem requires the shortest path between every pair of vertices u, v in the graph G . We can run Dijkstra’s algorithm [1] from all vertices which will have running time $O(mn + n^2 \log n)$, or use Floyd-Warshall algorithm [24], [25] with running time $O(n^3)$. Researchers have made many improvements since then [26], [27], [28], [29], [30], but there is still no truly subcubic time ($O(n^{3-\epsilon})$ for some constant $\epsilon > 0$) APSP algorithm for real-weighted graphs or even graphs with integer weights in $[0, n]$. Williams [31] gave an APSP algorithm with running time $n^3/2^{\Theta(\sqrt{\log n})}$ for real-weighted graphs. For undirected real-weighted graphs, Pettie and Ramachandran’s APSP algorithm [4] runs in $O(mn \log \alpha(m, n))$ time, and for directed real-weighted graphs, Pettie [32] gave an APSP algorithm in $O(mn + n^2 \log \log n)$ time.

II. PRELIMINARIES

In this paper we work on an undirected graph $G = (V, E, w)$ with vertex set V , edge set $E \subseteq V^2$ and non-negative weight function $w : E \rightarrow \mathbb{R}_{\geq 0}$, also denoted by w_{uv} . In an undirected graph $w_{uv} = w_{vu}$ holds for all edges $(u, v) \in E$. We denote $n = |V|$, $m = |E|$ as the number of vertices and edges in the graph, and $N(u) = \{v : (u, v) \in E\}$ as the set of neighbors of u . For two vertices $u, v \in V$, $\text{dist}_G(u, v)$ is length of the shortest path connecting u and v , namely the distance of u and v in graph G . The subscript G is omitted when the context is clear. Let s be the source vertex. The target of our algorithm is to find $\text{dist}(s, v)$ for every $v \in V$. Without loss of generality we assume that G is connected, so $m \geq n - 1$.

a) Constant-Degree Graph: Throughout the paper we need a graph with constant degree. To accomplish this, given a graph G , we construct G' by a classical transformation (see [33]):

- Substitute each vertex v with a cycle of $|N(v)|$ vertices x_{vw} ($w \in N(v)$) connected with undirected zero-weight edges, that is, for every neighbor w of v , there is a vertex x_{vw} on this cycle.
- For every edge (u, v) in G , add an undirected edge between corresponding vertices x_{uw} and x_{vu} with weight w_{uv} .

We can see distance $\text{dist}_{G'}(x_{uu'}, x_{vv'}) = \text{dist}_G(u, v)$ for arbitrary $u' \in N(u)$ and $v' \in N(v)$. Each vertex in G' has degree at most 3, while G' being a graph with $O(m)$ vertices and $O(m)$ edges.

b) Comparison-Addition Model: In this paper our algorithm works under *comparison-addition* model, in which real numbers are subject to only comparison and addition operations. In this model, each addition and comparison takes unit time, and no other computations on edge weights are allowed.

c) Fibonacci Heap: Under such a model, it is possible to construct a Fibonacci heap H that spends amortized $O(1)$ time for initialization, insertion, decrease-key operations, and

$O(\log |H|)$ time for each extract-min operation [2]. When we extract the minimum element from the heap, we also call that element “popped” from the heap.

III. MAIN ALGORITHM

In the following sections we assume that G is connected, and each vertex in G has degree no more than 3. (There are $O(m)$ vertices and $O(m)$ edges in G , but we still use $O(\log n)$ which is equivalent to $O(\log m)$ where n is the number of vertices in the original graph without degree constraints.)

Our algorithm is based on original Dijkstra’s algorithm [1]. Since the main bottleneck of Dijkstra’s algorithm is the $O(\log n)$ time per every vertex extracted from the Fibonacci heap [2], we only insert a subset $R \subseteq V$ of vertices into the heap. Each remaining vertex $v \in V \setminus R$ is *bundled* to its closest vertex in R . Throughout the algorithm, vertices are updated only when some vertex $u \in R$ is popped from the heap. Our algorithm consists of two stages: *bundle construction* and *Bundle Dijkstra*, whose details will be introduced in Section III-A and III-B respectively.

To demonstrate the main idea of our algorithm, in this section we first give an algorithm that runs in expected $O(m\sqrt{\log n \cdot \log \log n})$ time but not “with high probability”. In Section IV we give an improved version of the bundle construction stage, leading to an algorithm that runs in $O(m\sqrt{\log n \cdot \log \log n})$ time with high probability. Both algorithms always give correct answers.

A. Bundle Construction

A simple version of bundle construction works as follows¹ (k is a parameter to be determined later):

- Independently sample each vertex $v \in V \setminus \{s\}$ with probability $\frac{1}{k}$ to form set R , then add s into R .
- For each vertex $v \notin R$, run Dijkstra’s algorithm started from v until first vertex of R is extracted from the heap, denoted by $b(v)$. Therefore $b(v)$ is one of the closest vertices in R to v , i.e., $b(v) \in \arg \min_{u \in R} \text{dist}(u, v)$. We say that v is *bundled* to $b(v)$.
- For each $u \in R$, let $b(u) = u$, and $\text{Bundle}(u) = \{v : u = b(v)\}$ be the set of vertices bundled to u . By definition, $\{\text{Bundle}(u)\}_{u \in R}$ forms a partition of the vertex set V .
- For each vertex $v \notin R$, define $\text{Ball}(v) = \{w \in V : \text{dist}(v, w) < \text{dist}(v, b(v))\}$, that is, the set of vertices closer to v than its bundled vertex $b(v)$. In the previous Dijkstra’s algorithm we can get $\text{Ball}(v)$ and also values of $\text{dist}(v, w)$ for all $w \in \text{Ball}(v) \cup \{b(v)\}$.

a) Time Analysis of Bundle Construction: For each vertex $v \notin R$, without loss of generality we assume its Dijkstra’s algorithm breaks tie in a deterministic way. Therefore, the order of vertices extracted from heap is fixed.

¹One may notice that sampled set, closest sampled vertex and balls are common techniques in papers on shortest path algorithms, distance oracles and spanners, and there are deterministic construction algorithms for such “dominating set” (e.g. [34]), but the extra $O(\log n)$ factor for deterministic approaches introduced on the size of dominating set or construction time is not affordable here.

We can see $\mathbb{E}[|R|] = O(m/k)$. For each vertex $v \notin R$, let S_v be the set of vertices extracted before its Dijkstra’s algorithm stops, then $\text{Ball}(v) \subsetneq S_v$. By definition of R , $|S_v|$ follows geometric distribution with success probability $\frac{1}{k}$, thus $\mathbb{E}[|S_v|] = k$ and $\mathbb{E}[|\text{Ball}(v)|] \leq k$. By constant degree property, the number of vertices ever added into the heap is also $O(|S_v|)$, so the total time of the bundle construction is $O(\sum_{v \in V \setminus R} \mathbb{E}[|S_v| \log |S_v|]) = O(mk \log k)$ in expectation.

Remark 1. One may argue that $x \log x$ is a convex function so that $\mathbb{E}[|S_v| \log |S_v|] = O(k \log k)$ does not trivially hold due to Jensen’s inequality. Thanks to an anonymous reviewer, just notice that $\mathbb{E}[|S_v|^2] = 2k^2 - k = O(k^2)$ and $\sqrt{x} \log x$ is a concave function when $x \geq 1$.

B. Bundle Dijkstra

Given the set R and the partition of bundles, the main algorithm works as follows, with pseudocode given in Algorithm 1:

Initially we set $d(s) = 0$ and $d(v) = +\infty$ for all other vertex v , and insert all vertices of R into a Fibonacci heap [2]. Whenever we pop a vertex $u \in R$ from the heap, we update the distances by the following steps. (Here relaxing a vertex v by a value D means that we update $d(v)$ by $\min\{d(v), D\}$.)

- 1) For every vertex v bundled to u , we need to find the exact value of $\text{dist}(s, v)$. First relax v by $d(u) + \text{dist}(u, v)$; then for every vertex $y \in \text{Ball}(v)$, relax v by $d(y) + \text{dist}(y, v)$; and for every $z_2 \in \text{Ball}(v) \cup \{v\}$ and $z_1 \in N(z_2)$, relax v by $d(z_1) + w_{z_1, z_2} + \text{dist}(z_2, v)$. That is, we update $d(v)$ by its bundled vertex u , vertices in $\text{Ball}(v)$, and vertices neighboring to v and $\text{Ball}(v)$.
- 2) After updating $d(x)$ for every $x \in \text{Bundle}(u)$, we update the vertices $y \in N(x)$ and vertices $z_1 \in \text{Ball}(y)$. That is, relaxing y by $d(x) + w_{x, y}$ for all $y \in N(x)$ and then relaxing z_1 by $d(x) + w_{x, y} + \text{dist}(y, z_1)$ for all $z_1 \in \text{Ball}(y)$.
- 3) Whenever we update a vertex $v \notin R$, we also relax its bundled vertex $b(v)$ by $d(v) + \text{dist}(v, b(v))$. (But later we will see this is only needed in Step 2 but not Step 1, since in Step 1 v is bundled to u , but the distance $\text{dist}(s, u)$ is already found when popping u from the heap.)

The following observation holds naturally from the algorithm.

Observation 2. $d(v) \geq \text{dist}(s, v)$ always holds for all $v \in V$.

a) Time Analysis for Bundle Dijkstra: For the Bundle Dijkstra stage, only vertices in R are inserted into heap, thus the extract-min operation only takes $O(|R| \log n)$ time in total. Since every vertex in $V \setminus R$ only appears once as v and x in Step 1 and Step 2, respectively, and by constant degree property, every vertex appears constant times as the vertex $y \in N(x)$ in Step 2. We can see the number of vertices z_1, z_2 in Step 1 for every v is $O(|\text{Ball}(v)|)$, and the number of vertices z_1 in Step 2 for every y is $O(|\text{Ball}(y)|)$. Also note that the recursive call of RELAX in Step 3 can only recurse once since $b(v) \in R$. So the total time for Step 1, 2

Algorithm 1: BUNDLEDIJKSTRA(G, s)

Input : A graph $G = (V, E, w)$ and starting vertex $s \in V$

Output: Distance $d(v)$ from s to v for every vertex $v \in V$

```
1 Construct Bundles as described in Section III-A;
2 Set label  $d(s) \leftarrow 0$  and  $d(v) \leftarrow +\infty$  for all
    $v \in V \setminus \{s\}$ ;
3 Initialize Fibonacci heap  $H$  with all vertices of  $R$  and
   key  $d(\cdot)$ ;
4 while  $H$  is not empty do
5    $u \leftarrow H.EXTRACTMIN()$ ;
6   foreach  $v \in \text{Bundle}(u)$  do // Step 1
7     RELAX( $v, d(u) + \text{dist}(u, v)$ );
8     foreach  $y \in \text{Ball}(v)$  do
9       RELAX( $v, d(y) + \text{dist}(y, v)$ );
10    foreach  $z_2 \in \text{Ball}(v) \cup \{v\}$  do
11      foreach  $z_1 \in N(z_2)$  do
12        RELAX( $v, d(z_1) + w_{z_1, z_2} + \text{dist}(z_2, v)$ );
13  foreach  $x \in \text{Bundle}(u)$  do // Step 2
14    foreach  $y \in N(x)$  do
15      RELAX( $y, d(x) + w_{x, y}$ );
16      foreach  $z_1 \in \text{Ball}(y)$  do
17        RELAX( $z_1, d(x) + w_{x, y} + \text{dist}(y, z_1)$ );
18
19 function RELAX( $v, D$ ): // Step 3
20   if  $D < d(v)$  then
21      $d(v) \leftarrow D$ ;
22     if  $v \in H$  then
23        $H.DECREASEKEY(v, D)$ 
24     else if  $v \notin R$  then
25       RELAX( $b(v), d(v) + \text{dist}(v, b(v))$ )
```

and 3 is $O(\sum_{v \in V \setminus R} |\text{Ball}(v)|)$. Thus, the time of the bundle Dijkstra stage is $\mathbb{E}[O(|R| \cdot \log n + \sum_{v \in V \setminus R} |\text{Ball}(v)|)] = O(\frac{m}{k} \log n + mk)$ in expectation.

Now, we can see that the expected total time of the two stages is $O(\frac{m}{k} \log n + mk \log k)$, which is minimized to $O(m\sqrt{\log n \cdot \log \log n})$ if we choose $k = \sqrt{\frac{\log n}{\log \log n}}$. We move to explain our main ideas of the correctness proof. A formal proof is given in Section III-C.

b) Main ideas: The following propositions hold in the algorithm. (Here the iteration of u means the iteration performed when popping $u \in R$; a real distance $\text{dist}(s, v)$ is found means $d(v) = \text{dist}(s, v)$ already holds.)

Proposition 3. When popping $u \in R$ from the heap, its distance $\text{dist}(s, u)$ is already found.

Proposition 4. After Step 1 in the iteration of u , $\text{dist}(s, v)$ for all $v \in \text{Bundle}(u)$ are found.

The following lemmas contain the main ideas of the algo-

rithm.

Lemma 5. For any vertex $u \in R$ and any path P from s to u , if P goes through vertex y , $\text{dist}(s, b(y))$ is at most the length of P .

Proof. $\text{dist}(s, y)$ is at most the length of subpath of P from s to y . By definition of $b(y)$, $\text{dist}(y, b(y))$ is at most the length of subpath of P from y to u . Concatenating two subpaths together, $\text{dist}(s, b(y)) \leq \text{dist}(s, y) + \text{dist}(y, b(y))$ is at most the length of P . $\square$

Lemma 5 shows that for any vertex $u \in R$, the shortest path from s to u only contains vertices y with $\text{dist}(s, b(y)) \leq \text{dist}(s, u)$. This is the intuition why vertices of R are popped in increasing order of $\text{dist}(s, \cdot)$. However, the shortest path from s to some vertex $v \in \text{Bundle}(u)$ may go through some vertex y with $\text{dist}(s, b(y)) \geq \text{dist}(s, u)$, that is, $b(y)$ is still not popped from the heap. But surprisingly, with the ideas of Lemma 6 we can deal with this case even before the iteration of $b(y)$.

Lemma 6. For a vertex $v \notin R$, if the shortest path from s to v is shorter than $\text{dist}(s, b(v)) + \text{dist}(b(v), v)$, and it goes through a vertex y (other than v) such that $\text{dist}(s, b(y)) \geq \text{dist}(s, b(v))$, then on the shortest path from y to v there are two adjacent vertices z_1, z_2 such that $z_1 \in \text{Ball}(y) \cup \{y\}$ and $z_2 \in \text{Ball}(v) \cup \{v\}$.

Proof. We have $\text{dist}(y, v) = \text{dist}(s, v) - \text{dist}(s, y)$ and $\text{dist}(s, v) < \text{dist}(s, b(v)) + \text{dist}(b(v), v)$. By triangle inequality, $\text{dist}(s, y) \geq \text{dist}(s, b(y)) - \text{dist}(y, b(y))$, and by $\text{dist}(s, b(y)) \geq \text{dist}(s, b(v))$,

$$\begin{aligned} & \text{dist}(y, v) \\ & < \text{dist}(b(v), v) + \text{dist}(y, b(y)) + \text{dist}(s, b(v)) - \text{dist}(s, b(y)) \\ & \leq \text{dist}(b(v), v) + \text{dist}(y, b(y)). \end{aligned}$$

Let z_1 be the last vertex on the shortest path from y to v satisfying $\text{dist}(y, z_1) < \text{dist}(y, b(y))$, so $z_1 \in \text{Ball}(y)$. Then z_2 will be the next vertex after z_1 , so $\text{dist}(y, z_2) \geq \text{dist}(y, b(y))$, and $\text{dist}(z_2, v) < \text{dist}(v, b(v))$, so $z_2 \in \text{Ball}(v)$. (If $\text{dist}(y, b(y)) = 0$ then $z_1 = y$, and if $\text{dist}(y, v) < \text{dist}(y, b(y))$ then $z_2 = v$ and z_1 is the vertex before v .) $\square$

Then we can see Proposition 3 and 4 hold throughout the algorithm iteratively: (A formal proof will be given in Section III-C.)

- When we pop the source s from the heap, $d(s) = 0$, and the distances $\text{dist}(s, v)$ for all $v \in \text{Bundle}(s)$ are found in the bundle construction step, and can be put in $d(v)$ in Step 1.
- Assume Proposition 4 holds for the first i vertices popped, so the real distances for all vertices bundled to popped vertices are found after Step 1. By Step 2 and 3, we can see for all unpopped $u \in R$, the distance $\text{dist}(s, u)$ can be found if the shortest path from s to u does not go through vertices bundled to other unpopped vertices in the heap. If the next popped vertex u' does not satisfy

this, let y be the first vertex on the shortest path from s to u' which is bundled to an unpoped vertex $b(y)$ other than u' , so $\text{dist}(s, b(y))$ can be found. By Lemma 5, $\text{dist}(s, b(y)) \leq \text{dist}(s, u')$, so if $d(u') > \text{dist}(s, u')$, $b(y)$ will be the next popped vertex. Thus, $\text{dist}(s, u')$ for the next popped vertex u' is found before it is popped.

- If an unpoped vertex $u' \in R$ is updated in the iteration of popped vertex u , the new path to u' must go through a vertex in $\text{Bundle}(u)$. By Lemma 5, $d(u')$ cannot be updated to be smaller than $\text{dist}(s, u)$, so the unpoped vertices must have longer or equal distances than any popped vertex.
- Thus when popping a vertex $u \in R$, its distance $\text{dist}(s, u)$ is already found. For all vertex $v \in \text{Bundle}(u)$, if $\text{dist}(s, v)$ is not directly obtained by $d(u) + \text{dist}(u, v)$, that is, $\text{dist}(s, v) < \text{dist}(s, u) + \text{dist}(u, v)$, let x be the last vertex on the shortest path from s to v such that $b(x)$ is popped before u , and let y be the next vertex after x . We can see $\text{dist}(s, b(y)) \geq \text{dist}(s, u)$, so by Lemma 6, we get such z_1 and z_2 . Then from Proposition 4 $\text{dist}(s, x)$ can be found in Step 1 in the iteration of $b(x)$, then $\text{dist}(s, z_1)$ can be found in Step 2 of that iteration. In this iteration of u , $\text{dist}(s, v)$ can be set to $\text{dist}(s, z_1) + w_{z_1, z_2} + \text{dist}(z_2, v)$ in Step 1, so Proposition 4 still holds after this iteration.

C. Proof of Correctness

We give a formal proof based on the pseudocode of Algorithm 1. Define $u_i \in R$ as the vertex extracted in the i -th iteration of while-loop in Algorithm 1. Our key lemma in the following shows the main properties of the algorithm, therefore Bundle Dijkstra is correct no matter how R is chosen.

Lemma 7. *The following properties hold for any $i \geq 1$ in Bundle Dijkstra (Algorithm 1):*

- 1) When u_i is extracted from the heap, $d(u_i) = \text{dist}(s, u_i)$ holds.
- 2) After i -th iteration of the while-loop, $d(u) \geq d(u_i)$ for all $u \in R \setminus \{u_j\}_{j \leq i}$.
- 3) After Step 1 of i -th iteration of the while-loop, $d(v) = \text{dist}(s, v)$ for all $v \in \text{Bundle}(u_i)$.

Proof. We shall prove the lemma by induction on i .

The lemma holds for $i = 1$ since $d(s) = 0$ and $d(v) = \text{dist}(s, v)$ for all $v \in \text{Bundle}(s)$ after Line 7.

Suppose the lemma holds for every $i \leq t-1$, consider the case $i = t$.

- 1) Consider a shortest path P from s to u_t . Let x be the last vertex on P such that $x \in \text{Bundle}(u_j)$ for some $j < t$, and y be the next one after x , hence $y \in \text{Bundle}(u)$ for some $u \in R \setminus \{u_\ell\}_{\ell < t}$. By Property 3 of induction hypothesis $d(x) = \text{dist}(s, x)$ after Step 1 of j -th iteration. After that the algorithm updates $d(y)$ in Line 15 since $y \in N(x)$, and further $d(u)$. Therefore after $(t-1)$ -th iteration $d(y) = \text{dist}(s, x) + \text{dist}(x, y) = \text{dist}(s, y)$ and $d(u) \leq \text{dist}(s, y) + \text{dist}(y, u)$.

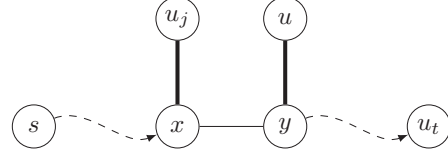


Fig. 1. Illustration of the shortest path from s to u_t .

Further:

$$\begin{aligned}
 d(u) &\leq \text{dist}(s, y) + \text{dist}(y, u) \\
 &\leq \text{dist}(s, y) + \text{dist}(y, u_t) \quad (b(y) = u) \\
 &= \text{dist}(s, u_t) \quad (y \text{ on shortest path}) \\
 &\leq d(u_t). \quad (\text{Observation 2})
 \end{aligned}$$

On the other hand, the algorithm extracts u_t from Fibonacci heap H immediately after $(t-1)$ -th iteration, thus $d(u_t) \leq d(u)$. So all the inequalities above should be equations, thus $d(u_t) = \text{dist}(s, u_t)$.

- 2) When executing Line 5 of t -th iteration, $d(u) \geq d(u_t)$ holds for every $u \in R \setminus \{u_j\}_{j \leq t}$ since H is a Fibonacci heap. Suppose $d(u) < d(u_t)$ for some $u \in R \setminus \{u_j\}_{j \leq t}$ after t -th iteration. The further updates on $d(u)$ must start from $d(x)$ for some $x \in \text{Bundle}(u_t)$. For last such update, applying Lemma 5 on this path from s to x then to u , we have:

$$\begin{aligned}
 d(u) &\geq \text{dist}(s, u_t) \quad (\text{Lemma 5}) \\
 &= d(u_t), \quad (\text{Property 1})
 \end{aligned}$$

leading to contradiction.

- 3) We want to show that $d(v) = \text{dist}(s, v)$ holds for all $v \in \text{Bundle}(u_t)$. Suppose there exists a vertex $v \in \text{Bundle}(u_t)$ such that $d(v) > \text{dist}(s, v)$ after Step 1. Denote P as the shortest path from s to v . Let x be the last vertex on P such that $x \in \text{Bundle}(u_j)$ for some $j < t$, and y be the next one after x on P , hence $y \in \text{Bundle}(u)$ for some $u \in R \setminus \{u_\ell\}_{\ell < t}$. By Property 3 of induction hypothesis, $d(x) = \text{dist}(s, x)$ after Step 1 of j -th iteration. Same as above we can show that $d(y) = \text{dist}(s, x) + \text{dist}(x, y) = \text{dist}(s, y)$ and $d(u) \leq \text{dist}(s, y) + \text{dist}(y, u)$ (where $u = b(y)$) before t -th iteration. We have:

$$\begin{aligned}
 \text{dist}(s, y) &\geq d(u) - \text{dist}(y, u). \\
 \text{dist}(s, v) &< d(v) \quad (\text{Assumption}) \\
 &\leq d(u_t) + \text{dist}(u_t, v). \\
 &\quad (d(v) \text{ updated in Line 7})
 \end{aligned}$$

On the other hand, $d(u) \geq d(u_t)$ after t -th iteration by Property 2. Since $d(u_t)$ doesn't change (Property 1 and Observation 2), and $d(u)$ can only decrease in t -th iteration, so $d(u) \geq d(u_t)$ holds throughout t -th iteration. Hence:

$$\begin{aligned}
 \text{dist}(y, v) &= \text{dist}(s, v) - \text{dist}(s, y) \\
 &< \text{dist}(u_t, v) + \text{dist}(y, u), \quad (1)
 \end{aligned}$$

while the equation holds since y lies on the shortest path from s to v .

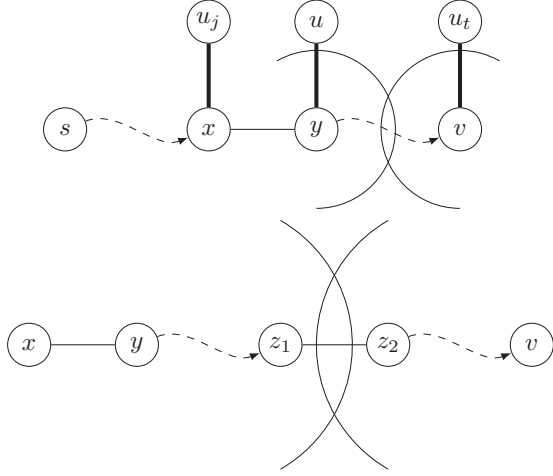


Fig. 2. Top: Illustration of the shortest path from s to v if $d(v) > \text{dist}(s, v)$. Bottom: A closer look at path x to v if $\text{dist}(y, v) \geq \text{dist}(u_t, v)$.

Therefore there are two possible cases:

- $\text{dist}(y, v) < \text{dist}(u_t, v)$.
In this case $y \in \text{Ball}(v)$, so we can update $d(v)$ to $\text{dist}(s, y) + \text{dist}(y, v)$ on Line 9, contradicting to $d(v) > \text{dist}(s, v)$.
- $\text{dist}(y, v) \geq \text{dist}(u_t, v)$.
First, by Inequality (1), $\text{dist}(y, u) > \text{dist}(y, v) - \text{dist}(u_t, v) \geq 0$. Let z_1 be the last vertex on path P with $\text{dist}(y, z_1) < \text{dist}(y, u)$, we have $z_1 \in \text{Ball}(y)$. Let z_2 be the next vertex on the path, then $\text{dist}(y, z_2) \geq \text{dist}(y, u)$, so $\text{dist}(z_2, v) = \text{dist}(y, v) - \text{dist}(y, z_2) \leq \text{dist}(y, v) - \text{dist}(y, u) < \text{dist}(u_t, v)$, that is, $z_2 \in \text{Ball}(v)$. (If z_2 does not exist, then $z_1 = v$.)
By Property 3 of induction hypothesis, $d(x) = \text{dist}(s, x)$ just after Step 1 of j -th iteration, so $d(z_1)$ is updated to $\text{dist}(s, z_1)$ in Line 17 of j -th iteration. Therefore $d(v)$ is updated to $\text{dist}(s, v)$ in Line 12 of t -th iteration (since $j < t$), contradicting the assumption.

Therefore $d(v) = \text{dist}(s, v)$ for all $v \in \text{Bundle}(u_t)$ after Step 1 of t -th iteration. $\square$

Remark 2. Pettie and Ramachandran [4] proved that, any hierarchy-based SSSP algorithm on undirected graphs in comparison-addition model takes time at least $\Omega(m + \min\{n \log \log r, n \log n\})$, where r is the ratio of the maximum-to-minimum edge weight. This bound becomes $\Omega(m + n \log n)$ when r is exponentially large. Here the hierarchy-based algorithm is defined to generate a permutation π_s satisfying *hierarchy property*: $\text{dist}(s, v) \geq \text{dist}(s, u) + \text{sep}(u, v) \Rightarrow \pi_s(u) < \pi_s(v)$, where $\text{sep}(u, v)$ is the longest edge on the MST path between u and v . The permutation

π_s is, though not defined to be, typically for the algorithms discussed in [4], the order that the algorithm visits the vertices. However, that is a worst-case lower bound, and our algorithm is randomized. Also the order that our algorithm visits the vertices *does not* follow the hierarchy property: think of two vertices x and y are both connected to u by edges $(x, u), (y, u)$ both with weight 1, and x, y are both bundled to u . It is possible that $\text{sep}(x, y) = 1$ and $\text{dist}(s, y) = \text{dist}(s, x) + 2$ but we set no limit on the order we visit x and y , that it is possible we visit y before x . This explains why our algorithm can break this $\Omega(m + n \log n)$ lower bound in [4].

IV. IMPROVED BUNDLE CONSTRUCTION

In this section we propose an improved bundle construction that runs in $O(m\sqrt{\log n \cdot \log \log n})$ time with high probability. In Section III-C we showed that correctness of Bundle Dijkstra does not depend on the choice of R , as long as $\text{b}(\cdot), \text{Ball}(\cdot)$ and $\text{Bundle}(\cdot)$ are correctly computed with respect to R . The running time for the bundle construction is $O(\sum_{v \in V \setminus R} |S_v| \log |S_v|)$, and Bundle Dijkstra is $O(\sum_{v \in V \setminus R} |\text{Ball}(v)| + |R| \log n)$.

Naturally, $|S_v|$ is a random variable following geometric distribution for each vertex $v \in V$, and they are not independent since a vertex $x \in V$ may appear in several sets. However, for a subset $W \subseteq V$, if any vertex appears at most once in $\{S_v\}_{v \in W}$, the corresponding random variables $\{|S_v|\}_{v \in W}$ are independent. By Lemma 9, if each random variable is dependent to few other variables, their summation deviates from the expectation with exponentially small probability. So we manually include all those vertices with $|S_v| \geq k \log k$ into R . In this way for each vertex in $V \setminus R$, its random variable is dependent to only a limited number of other ones, and we can bound their summation with high probability.

We introduce how to generate R and compute $\{\text{b}(v)\}_{v \in V \setminus R}$ below, as well as $\{\text{Ball}(v)\}_{v \in V \setminus R}$, $\{\text{Bundle}(u)\}_{u \in R}$ and $\text{dist}(v, u)$ for $u \in \text{Ball}(v) \cup \{\text{b}(v)\}$. The pseudocode is given in Algorithm 2. We still set parameter $k = \sqrt{\frac{\log n}{\log \log n}}$ as in Section III.

a) Improved Bundle Construction:

- Sample each vertex $v \in V \setminus \{s\}$ with probability $\frac{1}{k}$ to form set R_1 and add s into R_1 ;
- For each $v \in V \setminus R_1$, run Dijkstra algorithm started from v , until we have extracted a vertex in R_1 ; or have already popped $k \log k$ vertices.
- In the former case, denote the extracted vertices in the order they appeared as list $V_{\text{extract}}^{(v)}$. Note that $V_{\text{extract}}^{(v)}$ is similar to S_v of Section III. In the latter case, add v into R_2 ;
- Set $R = R_1 \cup R_2$, and for $v \in V \setminus R$, let the first vertex in $V_{\text{extract}}^{(v)}$ that lies in R be $\text{b}(v)$;
- With the results above, compute $\text{Bundle}(u)$ for $u \in R$, $\text{Ball}(v)$ for $v \in V \setminus R$, and record $\text{dist}(v, u)$ for $u \in \text{Ball}(v) \cup \{\text{b}(v)\}$. This step takes linear time.

The correctness of this bundle construction follows from the Dijkstra's algorithm [1]. We only need to analyze

$|R|$, $\sum_{v \in V \setminus R} |\text{Ball}(v)|$ and its running time. The performance of this improved bundle construction is characterized in Lemma 8 below. By Lemma 8, the bundle construction takes $O(mk \log k)$ time, and bundle Dijkstra takes $O(\sum_{v \in V \setminus R} |\text{Ball}(v)| + |R| \log n) = O(mk + m \log n/k)$ with probability $1 - e^{-\Omega(n^{1-o(1)})}$. Thus the total running time of our algorithm is $O(mk \log k + m \log n/k) = O(m\sqrt{\log n \cdot \log \log n})$ w.h.p. The proof of Lemma 8 is based on Lemma 9.

Algorithm 2: BUNDLECONSTRUCTION(G, s, k)

Input : A graph $G = (V, E, w)$, the source vertex $s \in V$ and parameter k

Output: All R , $b(\cdot)$, $\text{Ball}(\cdot)$, $\text{Bundle}(\cdot)$ as described above

Initialize $R_1 \leftarrow \{s\}$, and insert each $v \in V \setminus \{s\}$ to R_1 independently with probability $\frac{1}{k}$;

Initialize $R_2 \leftarrow \emptyset$;

foreach $v \in V \setminus R_1$ **do** // Truncated Dijkstra

Initialize Fibonacci heap $H^{(v)}$ with vertex v and its key $d^{(v)}(v) \leftarrow 0$;

Initialize an ordered list $V_{\text{extract}}^{(v)} \leftarrow ()$;

while $H^{(v)}$ is not empty **do**

$u \leftarrow H.\text{EXTRACTMIN}()$;

$V_{\text{extract}}^{(v)}.\text{APPEND}(u)$;

if $u \in R_1$ **then** quit the **while-loop**;

else if $|V_{\text{extract}}^{(v)}| > k \log k$ **then** set

$R_2 \leftarrow R_2 \cup \{v\}$ and quit the **while-loop**;

else

foreach $x \in N(u)$ **do**

if $x \notin H^{(v)}$ and $x \notin V_{\text{extract}}^{(v)}$ **then**

$H^{(v)}.\text{INSERT}(x, d^{(v)}(u) + w_{ux})$;

else if $d^{(v)}(x) > d^{(v)}(u) + w_{ux}$ **then**

$H.\text{DECREASEKEY}(x, d^{(v)}(u) + w_{ux})$;

$R \leftarrow R_1 \cup R_2$;

foreach $v \in V \setminus R$ **do**

$b(v) \leftarrow$ the first vertex in the ordered list $V_{\text{extract}}^{(v)}$ that lies in R ;

Compute $\{\text{Bundle}(u)\}_{u \in R}$, $\{\text{Ball}(v)\}_{v \in V \setminus R}$ and record $\text{dist}(v, u)$ for $u \in \text{Ball}(v) \cup \{b(v)\}$;

Lemma 8. By running Algorithm 2, with probability $1 - e^{-\Omega(n^{1-o(1)})}$, the following properties hold:

- (a) $|R| = O(\frac{m}{k})$.
- (b) $\sum_{v \in V \setminus R} |\text{Ball}(v)| = O(mk)$.
- (c) The running time of Algorithm 2 is $O(mk \log k)$.

Proof. First, each vertex of $V \setminus \{s\}$ is inserted to R_1 independently with probability $\frac{1}{k}$, so by Chernoff bound, with probability $1 - O(e^{-m/k}) = 1 - e^{-\Omega(n^{1-o(1)})}$, $|R_1| = \Theta(m/k)$, and meanwhile $m' := |V \setminus R_1| = \Theta(m)$.

For each vertex $v \in V \setminus R_1$, define $X_v = \mathbb{I}[v \in R_2]$ and $Y_v = |V_{\text{extract}}^{(v)}|$. Then, X_v is a Bernoulli random variable, $X_v \in [0, 1]$ with probability 1 and $\mathbb{E}[X_v] = (1 - \frac{1}{k})^{k \log k} = \Theta(\frac{1}{k})$. And Y_v is a geometric random variable except its value is zero when $X_v = 1$, so $Y_v \in [0, k \log k]$ with probability 1 and $\mathbb{E}[Y_v] = k - (k + k \log k)(1 - \frac{1}{k})^{k \log k} = \Theta(k)$.²

For each vertex $v \in V \setminus R_1$, denote $V_{\text{full}}^{(v)}$ as the first $k \log k$ vertices extracted in the Dijkstra algorithm if it did not truncate. They are determined by the structure of G , so there is no randomness in $V_{\text{full}}^{(v)}$. The values of X_v and Y_v are determined by whether vertices in $V_{\text{full}}^{(v)}$ were inserted into R_1 . Therefore, if $V_{\text{full}}^{(v_1)}, V_{\text{full}}^{(v_2)}, \dots, V_{\text{full}}^{(v_j)}$ are disjoint, then $X_{v_1}, X_{v_2}, \dots, X_{v_j}$ are independent, and similarly, $Y_{v_1}, Y_{v_2}, \dots, Y_{v_j}$ are independent.

For each vertex $w \in V_{\text{full}}^{(v)}$, because w is found by v within $k \log k$ steps of Dijkstra's algorithm, there must exist a path from v to w of no more than $k \log k$ edges, so by constant degree property, there are at most $3 \cdot (1 + 2 + \dots + 2^{k \log k - 1}) \leq 3 \cdot 2^{k \log k}$ different u such that $w \in V_{\text{full}}^{(u)}$. Hence, for each v , there are at most $3k \log k \cdot 2^{k \log k} = O(n^{o(1)})$ different $u \in V \setminus R_1$ such that $V_{\text{full}}^{(v)} \cap V_{\text{full}}^{(u)} \neq \emptyset$.

To apply Lemma 9, for each $v \in R_1$, also define X_v, Y_v and $V_{\text{full}}^{(v)}$ in the same way as if v is executed in the loop of Algorithm 2.

Now, we apply Lemma 9 for $\{X_v\}_{v \in V}$ and $\{Y_v\}_{v \in V}$. For $\{X_v\}_{v \in V}$, S is V and $|V| = m$, $\mu = \Theta(\frac{1}{k})$, $b = 1$, $T = O(n^{o(1)})$, and $\{W_v\}_{v \in V}$ are $\{V_{\text{full}}^{(v)}\}_{v \in V}$, and we can verify that $8Tb\mu^{-1} = O(n^{o(1)})$ and $8b^3T/\mu^3 = O(n^{o(1)})$, so with probability at least $1 - e^{-\Omega(m/n^{o(1)})}$, it holds that $\sum_{v \in S} X_v = \Theta(m/k)$. And for $\{Y_v\}_{v \in V}$, S is V , $\mu = \Theta(k)$, $b = k \log k$, $T = O(n^{o(1)})$, and $\{W_v\}_{v \in V}$ are also $\{V_{\text{full}}^{(v)}\}_{v \in V}$, and similarly we infer that with probability $1 - e^{-\Omega(m/n^{o(1)})}$, it holds that $\sum_{v \in S} Y_v = \Theta(mk)$. Thus, we conclude that with probability $1 - e^{-\Omega(n^{1-o(1)})}$, $\sum_{v \in V \setminus R_1} X_v = \Theta(m/k)$ and $\sum_{v \in V \setminus R_1} Y_v = \Theta(mk)$.

Then, we prove the three claims of this lemma.

For (a), by definition $|R| = |R_1| + |R_2|$, so by union bound, with probability $1 - e^{-\Omega(n^{1-o(1)})}$, $|R| = |R_1| + \sum_{v \in V \setminus R_1} X_v = O(\frac{m}{k})$.

For (b), by definition $|\text{Ball}(v)| \leq Y_v$, so $\sum_{v \in V \setminus R} |\text{Ball}(v)| \leq \sum_{v \in V \setminus R_1} Y_v$. Thus, with probability $1 - e^{-\Omega(n^{1-o(1)})}$, $\sum_{v \in V \setminus R} |\text{Ball}(v)| = O(mk)$.

For (c), we count the total time for the truncated Dijkstra algorithm in all iterations. For each vertex $v \in V \setminus R_1$, by constant degree property, the number of INSERT operations is $O(Y_v)$, so $|H^{(v)}| = O(Y_v) = O(k \log k)$. Therefore, each EXTRACTMIN operation takes time $O(\log(Y_v)) = O(\log k)$, and every other operation takes constant time. Thus the truncated Dijkstra algorithm of v takes time $O(Y_v \log k)$. Thus,

²Detailed calculation: $\mathbb{E}[Y_v] = k - \sum_{i=k \log k+1}^{+\infty} \frac{1}{k} (1 - \frac{1}{k})^{i-1} \cdot i = k - (1 - \frac{1}{k})^{k \log k} \sum_{i=1}^{+\infty} \frac{1}{k} (1 - \frac{1}{k})^{i-1} \cdot (i + k \log k) = k - (1 - \frac{1}{k})^{k \log k} (k + k \log k)$. Noticing that $(1 - \frac{1}{k})^{k \log k} \leq 1/k$, so $k - 1 - \log k \leq \mathbb{E}[Y_v] \leq k$ and $\mathbb{E}[Y_v] = \Theta(k)$.

with probability $1 - e^{-\Omega(n^{1-o(1)})}$, the total time of Algorithm 2 is $O(\sum_{v \in V \setminus R_1} Y_v \log k) = O(mk \log k)$. $\square$

Lemma 9. (Similar arguments as in [35]) Suppose a set of random variables $\{Z_v\}_{v \in S}$ satisfy that for each $v \in S$, $\mathbb{E}[Z_v] = \mu$, $Z_v \in [0, b]$ with probability 1, and each Z_v is corresponded to a fixed deterministic set W_v such that, if $W_{v_1}, W_{v_2}, \dots, W_{v_j}$ are disjoint, then $Z_{v_1}, Z_{v_2}, \dots, Z_{v_j}$ are independent, and W_v intersects with at most T different other W_u .

Then, with probability at least $1 - 8Tb\mu^{-1} \cdot e^{-\frac{\mu^3|S|}{8b^3T}}$, it holds that $\sum_{v \in S} Z_v = \Theta(|S|\mu)$.

Proof. We try to partition $\{Z_v\}_{v \in S}$ into several subsets $\{Z_t\}$ such that all Z_v 's in each Z_t are independent so that we can apply Hoeffding's inequality, or the size of Z_t is small so that we can bound them by the upper bound b , and finally combine everything by the union bound. Also note that we do not need to actually compute $\{Z_t\}$, as they are merely introduced for this mathematical proof.

Fix parameter $p = \frac{|S|\mu}{4Tb}$. Since each W_v intersects with at most T different other W_u , whenever there are at least $p(T+1)$ elements in $\{Z_v\}_{v \in S}$, we can pick p different Z_v from them whose W_v are disjoint, so that they are independent: pick an arbitrary Z_v and discard those Z_u if $W_u \cap W_v \neq \emptyset$; since there are at most T such Z_u different from Z_v , every time we discard at most $T+1$ elements. So from $p(T+1)$ elements we can pick p of them. We let them form a Z_t and remove them from $\{Z_v\}_{v \in S}$. Repeating this process we will end up with a partition $\{Z_1, Z_2, \dots, Z_q, Z_{q+1}\}$ of $\{Z_v\}_{v \in S}$ such that: $|Z_t| = p$, and all $Z_v \in Z_t$ are independent for $1 \leq t \leq q$; $|Z_{q+1}| \leq p(T+1) \leq 2pT = \frac{\mu}{2b}|S|$. By definition $\mu \leq b$, so $|Z_{q+1}| \leq \frac{1}{2}|S|$.

Then by Hoeffding's inequality, for each $1 \leq t \leq q$,

$$\Pr \left[\left| \sum_{v \in Z_t} Z_v - |Z_t|\mu \right| \geq \frac{1}{2}|Z_t|\mu \right] \leq 2e^{-\frac{2(\frac{1}{2}|Z_t|\mu)^2}{|Z_t|b^2}} = 2e^{-\frac{\mu^2 p}{2b^2}}.$$

and $0 \leq \sum_{v \in Z_{q+1}} Z_v \leq |Z_{q+1}|b$ with probability 1.

By union bound, with probability at least $1 - 2qe^{-\frac{\mu^2 p}{2b^2}}$,

$$\begin{aligned} \sum_{v \in S} Z_v &\geq \frac{1}{2} \sum_{t=1}^q |Z_t|\mu = \frac{1}{2}(|S| - |Z_{q+1}|)\mu \\ &\geq \frac{1}{2} \left(|S| - \frac{1}{2}|S| \right) \mu = \frac{1}{4}|S|\mu, \end{aligned}$$

and meanwhile

$$\sum_{v \in S} Z_v \leq \frac{3}{2} \sum_{t=1}^q |Z_t|\mu + |Z_{q+1}|b \leq \frac{3}{2}|S|\mu + \frac{\mu}{2}|S| = 2|S|\mu.$$

And from $|S| \geq \sum_{t=1}^q |Z_t| \geq qp$, we conclude that $q \leq |S|/p = 4Tb/\mu$. Thus, with probability at least $1 - 8Tb\mu^{-1}e^{-\frac{\mu^3|S|}{8b^3T}}$, it holds that $\sum_{v \in S} Z_v = \Theta(|S|\mu)$. $\square$

V. DISCUSSION

We gratefully acknowledge an anonymous reviewer for suggesting that constant-degree is not a necessary condition for this algorithm, so we can get improved time complexity when $m = \omega(n)$ and $m = o(n \log n)$. Instead of making the graph of degree 3, we use similar methods to split the vertices of degree $> m/n$ to vertices of degrees $\leq m/n$, so that the number of vertices is still $O(n)$. Then in each step:

- In bundle construction, the time for Dijkstra search for every vertex v will become $O(|S_v| \cdot \frac{m}{n} + |S_v| \log(|S_v| \cdot \frac{m}{n}))$, since the size of the heap is at most $|S_v| \cdot \frac{m}{n}$, so in total $O(mk + nk \log(mk/n))$.
- The time for Bundle Dijkstra will become $O(\frac{n}{k} \log n + mk)$, since the number of vertices z_1 in Step 1 for every v is $O(\frac{m}{n} |\text{Ball}(v)|)$, and the number of vertices z_1 in Step 2 for every y is $O(|\text{Ball}(y)|)$ but each vertex appears $O(m/n)$ times as y in Step 2.
- When $m/n = o(\log n)$, one can check that the analysis of independence in Section IV still works, since the number of different $u \in V \setminus R_1$ which have $V_{\text{full}}^{(v)} \cap V_{\text{full}}^{(u)} \neq \emptyset$ for each $v \in V \setminus R_1$ is still $O(n^{o(1)})$.

Thus, the time complexity for this algorithm is $O(\frac{n}{k} \log n + mk + nk \log(mk/n))$. When $m < n \log \log n$, k still equals to $\sqrt{\frac{\log n}{\log \log n}}$, and the time bound is $O(n\sqrt{\log n \log \log n})$. When $n \log \log n \leq m < n \log n$, let $k = \sqrt{\frac{n}{m} \log n}$, and the time bound will be $O(\sqrt{mn \log n})$.

REFERENCES

- [1] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [2] M. L. Fredman and R. E. Tarjan, "Fibonacci heaps and their uses in improved network optimization algorithms," *JACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [3] J. R. Driscoll, H. N. Gabow, R. Shrairman, and R. E. Tarjan, "Relaxed heaps: An alternative to fibonacci heaps with applications to parallel computation," *Commun. ACM*, vol. 31, no. 11, p. 1343–1354, Nov 1988.
- [4] S. Pettie and V. Ramachandran, "A shortest path algorithm for real-weighted undirected graphs," *SIAM Journal on Computing*, vol. 34, no. 6, pp. 1398–1431, 2005.
- [5] A. C. C. Yao, "An $O(|E| \log \log |V|)$ algorithm for finding minimum spanning trees," *Information Processing Letters*, vol. 4, no. 1, pp. 21–23, 1975.
- [6] D. R. Karger, P. N. Klein, and R. E. Tarjan, "A randomized linear-time algorithm to find minimum spanning trees," *J. ACM*, vol. 42, no. 2, p. 321–328, Mar 1995.
- [7] B. Chazelle, "A minimum spanning tree algorithm with inverse-ackermann type complexity," *J. ACM*, vol. 47, no. 6, p. 1028–1047, Nov 2000.
- [8] S. Pettie and V. Ramachandran, "An optimal minimum spanning tree algorithm," *J. ACM*, vol. 49, no. 1, p. 16–34, Jan 2002.
- [9] H. N. Gabow and R. E. Tarjan, "Algorithms for two bottleneck optimization problems," *Journal of Algorithms*, vol. 9, no. 3, pp. 411–417, 1988.
- [10] S. Chechik, H. Kaplan, M. Thorup, O. Zamir, and U. Zwick, "Bottleneck paths and trees and deterministic graphical games," in *STACS*, ser. LIPIcs, N. Ollinger and H. Vollmer, Eds., vol. 47. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2016, pp. 27:1–27:13.
- [11] R. Duan, K. Lyu, H. Wu, and Y. Xie, "Single-source bottleneck path algorithm faster than sorting for sparse graphs," *CoRR*, vol. abs/1808.10658, 2018.
- [12] V. V. Williams, "Nondecreasing paths in a weighted graph or: How to optimally read a train schedule," *ACM Trans. Algorithms*, vol. 6, no. 4, sep 2010.

- [13] M. L. Fredman and D. E. Willard, "Surpassing the information theoretic bound with fusion trees," *Journal of Computer and System Sciences*, vol. 47, no. 3, pp. 424–436, 1993.
- [14] M. L. Fredman and D. E. Willard, "Trans-dichotomous algorithms for minimum spanning trees and shortest paths," *Journal of Computer and System Sciences*, vol. 48, no. 3, pp. 533–551, 1994.
- [15] M. Thorup, "Floats, integers, and single source shortest paths," *J. Algorithms*, vol. 35, no. 2, p. 189–201, May 2000.
- [16] R. Raman, "Priority queues: Small, monotone and trans-dichotomous," in *Proceedings of the Fourth Annual European Symposium on Algorithms*, ser. ESA '96. Berlin, Heidelberg: Springer-Verlag, 1996, p. 121–137.
- [17] R. Raman, "Recent results on the single-source shortest paths problem," *SIGACT News*, vol. 28, no. 2, p. 81–87, Jun 1997.
- [18] M. Thorup, "On ram priority queues," *SIAM Journal on Computing*, vol. 30, no. 1, pp. 86–109, 2000.
- [19] T. Hagerup, "Improved shortest paths on the word ram," in *Automata, Languages and Programming*, U. Montanari, J. D. P. Rolim, and E. Welzl, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2000, pp. 61–72.
- [20] M. Thorup, "Undirected single-source shortest paths with positive integer weights in linear time," *J. ACM*, vol. 46, no. 3, p. 362–394, May 1999.
- [21] M. Thorup, "Integer priority queues with decrease key in constant time and the single source shortest paths problem," *Journal of Computer and System Sciences*, vol. 69, no. 3, pp. 330–353, 2004, special Issue on STOC 2003.
- [22] L. Chen, R. Kyng, Y. P. Liu, R. Peng, M. P. Gutenberg, and S. Sachdeva, "Maximum flow and minimum-cost flow in almost-linear time," in *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, 2022, pp. 612–623.
- [23] A. Bernstein, D. Nanongkai, and C. Wulff-Nilsen, "Negative-weight single-source shortest paths in near-linear time," in *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, 2022, pp. 600–611.
- [24] R. W. Floyd, "Algorithm 97: Shortest path," *Communications of the ACM*, vol. 5, p. 345, 1962.
- [25] S. Warshall, "A theorem on boolean matrices," *J. ACM*, vol. 9, no. 1, p. 11–12, Jan 1962.
- [26] M. L. Fredman, "New bounds on the complexity of the shortest path problem," *SIAM Journal on Computing*, vol. 5, no. 1, pp. 83–89, 1976.
- [27] W. Dobosiewicz, "A more efficient algorithm for the min-plus multiplication," *International Journal of Computer Mathematics*, vol. 32, no. 1–2, pp. 49–60, 1990.
- [28] U. Zwick, "A slightly improved sub-cubic algorithm for the all pairs shortest paths problem with real edge lengths," in *Algorithms and Computation: 15th International Symposium, ISAAC 2004, Hong Kong, China, December 20–22, 2004. Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2004, p. 921–932.
- [29] T. M. Chan, "All-pairs shortest paths with real weights in $O(n^3/\log n)$ time," *Algorithmica*, vol. 50, no. 2, pp. 236–243, 2008.
- [30] Y. Han and T. Takaoka, "An $O(n^3 \log \log n / \log^2 n)$ time algorithm for all pairs shortest paths," in *Proceedings of the 13th Scandinavian Conference on Algorithm Theory*, ser. SWAT'12. Berlin, Heidelberg: Springer-Verlag, 2012, p. 131–141.
- [31] R. Williams, "Faster all-pairs shortest paths via circuit complexity," in *Proceedings of the Forty-Sixth Annual ACM Symposium on Theory of Computing*, ser. STOC '14. New York, NY, USA: Association for Computing Machinery, 2014, p. 664–673.
- [32] S. Pettie, "A new approach to all-pairs shortest paths on real-weighted graphs," *Theoretical Computer Science*, vol. 312, no. 1, pp. 47–74, 2004, automata, Languages and Programming.
- [33] G. N. Frederickson, "Data structures for on-line updating of minimum spanning trees," in *Proceedings of the Fifteenth Annual ACM Symposium on Theory of Computing*, ser. STOC '83. New York, NY, USA: Association for Computing Machinery, 1983, p. 252–257.
- [34] D. Aingworth, C. Chekuri, and R. Motwani, "Fast estimation of diameter and shortest paths (without matrix multiplication)," in *Proceedings of the Seventh Annual ACM-SIAM Symposium on Discrete Algorithms*, ser. SODA '96. USA: Society for Industrial and Applied Mathematics, 1996, p. 547–553.
- [35] S. Janson, "Large deviations for sums of partly dependent random variables," *Random Struct. Algorithms*, vol. 24, no. 3, p. 234–248, May 2004.